

CS2106 Cheatsheet, AY26/27 S1

Zhao Wu and Tien Cheng

August 30, 2026

1 Introduction

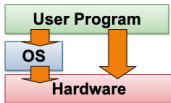
What is OS?

OS is a program that acts as an **intermediary** between a **computer user** and the **computer hardware**. (Simplified definition)

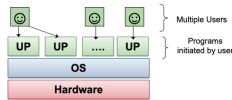
- E.g.
 - Windows, macOS, Linux, Solaris, FreeBSD
 - iOS, Android
 - PS5, Xbox, Nintendo Switch
 - Smart TV, Smart Watch

Brief History of OS

- First computer:
 - OS Type:** No OS
 - Program directly interacts with hardware.
 - Reprogram by physically changing the hardware configuration (cables, switches, punched paper tape)
 - Advantage:**
 - Minimal overhead due to OS
 - Disadvantage:**
 - Not portable (to port, need to manually rewrite the program)
 - Inefficient use of the computer
 - E.g. Electronic Numerical Integrator And Computer (ENIAC), Harvard Mark I
- Mainframes
 - Characteristics:**
 - No interactive interface (program via paper tape, magnetic tape, punch card)
 - Batch Processing Only
 - OS Type:** Batch OS
 - User still interacts with hardware directly
 - Additional information for OS (e.g. resource required, job specification)
 - Advantage:**
 - Execute user programs (i.e. jobs) one at a time
 - Disadvantage:**
 - Simple batch processing is inefficient as the CPU is idle during I/O.
 - Multiprogramming is absent.
 - E.g. IBM 360



- Time-Sharing OS
 - OS Type:** Time-Sharing OS
 - Advantage:**
 - Allow multiple users to interact with the machine using terminals (teletypes)
 - User job scheduling (illusion of concurrency)
 - OS manages CPU time, memory and storage
 - Virtualization of hardware (each program executes as if it has all the resources to itself)
 - E.g. Apple II PC, IBM PC



- Personal OS
 - Windows model:
 - Single user at a time, but possibly more than 1 user can access
 - General time-sharing model
 - Unix model:
 - One user at the workstation, but other users can access remotely

- General time-sharing model

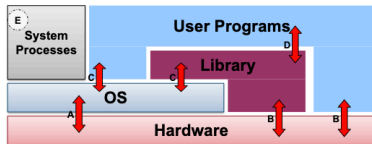
1.1 Motivation of OS

- Abstraction**
 - Hide low-level details and present higher-level functionality to the user
 - Motivation:
 - Large variation in hardware configuration, but the same hardware has well-defined functionality. (E.g. hard disk rotation speed varies, but it can store and retrieve information)
 - Efficiency, Programmability and Portability
- Resource Allocator**
 - Manage all resources (CPU, Memory, I/O) and arbitrate potentially conflicting requests for efficient and fair resource use
 - Motivation:
 - Program requires multiple hardware resources to run.
 - Multiple programs should run simultaneously for better utilisation
- Control Program**
 - Control execution of programs to:
 - Prevent errors and improper use of the computer
 - Motivation:
 - Program may "misuse" the computer
 - Bugs (accident), Virus, Malware (malicious)
 - Multiple users can share the computer
 - Security, isolation and protection

1.2 OS Structures

High-Level View of OS

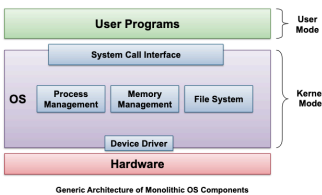
- OS is software that runs in **Kernel Mode** (i.e. direct access to all hardware resources)
 - Can't use system calls in kernel code
 - Can't use normal libraries and I/O
- Other software operates in **User Mode** (i.e. limited/controlled access to hardware resources)



- A:** OS executing machine instructions
- B:** Normal machine instructions executed (program/library code)
- C:** Calling OS using **system call interface**
- D:** User program calls library code
- E:** System processes
 - Provide high-level services, usually part of OS

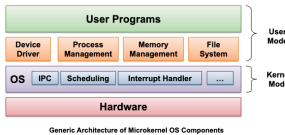
OS Structures

- Monolithic**
 - Big program (e.g. Linux source code)
 - If Kernel fails, BSOD
 - Better performance



2. Microkernel

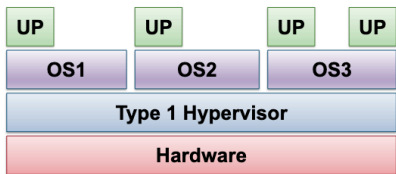
- Smaller and cleaner abstraction
- Provides basic and essential facilities:
 - Inter-Process Communication
 - Address space management
 - Thread management
- Higher-level OS services are run outside of the kernel, using IPC to communicate.
- Lower performance



1.3 Virtual Machines

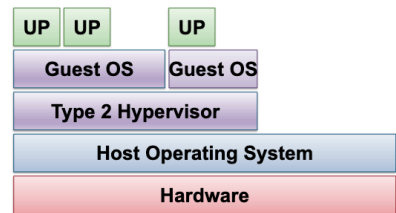
A **Virtual Machine** emulates the underlying hardware and is created and managed by a **Hypervisor** (aka **Virtual Machine Monitor (VMM)**)

- Type 1 Hypervisor



- Type 1 hypervisor:**
 - AKA bare-metal hypervisor
 - Provides individual **virtual machines** to guest OSES
 - E.g., IBM VM/370, VMware ESXi

- Type 2 Hypervisor



- Type 2 hypervisor**
 - Runs in host OS
 - Guest OS runs inside Virtual Machine
 - e.g., VMware Workstation, VirtualBox, QEMU

2 Process Abstraction

Motivation

- Allowing only one program to run at a time is inefficient, so the OS enables multiple programs to run concurrently.
- To switch between programs, the OS needs to save the context of the current program and load the context of the next program.
- A **process** (or task or job) is the OS abstraction for a running program.

2.1 Process Context

To allow multiple programs to run concurrently, the OS manages the following components:

Layer	Component	Stores
Memory	Text	program instructions
	Data	global variables and static variables
	Stack	collection of stack frames (calls + locals)
	Heap	region of memory used to store dynamically allocated data
Hardware	General-Purpose Registers (GPRs)	temporary data
	Program Counter (PC)	next instruction address
	Stack Pointer (SP)	top of stack frame address
	Frame Pointer (FP)	fixed location in stack frame address
OS	Process ID (PID)	unique process ID
	Process State	current process state

2.2 Stack Memory & Function Calls

Stack Memory

- The **stack** is a region of memory used to support function calls.
- Each function invocation **pushes a new stack frame onto the stack**, while each function return **pops the top stack frame**.
- Each stack frame contains (CS2106 convention, caller-pushed first):

Component	Who	Why
Local variables	Callee	Callee's own local variables
Parameters (only those that don't fit in registers)	Caller	so the callee can read the parameters of function call
Saved GPRs	Callee	Copy of GPRs which callee modifies; protect the caller's values across the call
Saved old SP	Callee	restore the caller's SP
Saved old FP	Callee	restore the caller's FP
Return address / Saved PC	Caller	return to the right instruction after the callee returns

- The stack pointer (SP) points to the top of the stack
- The frame pointer (FP) points to the base of the current stack frame
- We use a frame pointer because the stack pointer tracks the current top of the stack, which can move as local variables are pushed and popped. The FP is fixed, so we can use a fixed offset to access arguments and local variables.
- Restoring the SP does not erase the stack frame, so always initialise local variables as uninitialised local variables can contain leftover values from previous calls.

Function Call Convention

Function calling convention differs depending on the hardware and software. The following is an example of such convention in 2106:

- Function Call Preparation:**
 - Caller: pass parameters using registers and/or the stack
 - Caller: save return program counter onto the stack
- Transfer of Control from Caller to Callee:**
 - Callee: **save registers used by the callee, save the old frame pointer**, save the old stack pointer
 - Callee: allocate space for local variables of the callee on the stack
 - Callee: adjust the stack pointer to point to the new stack top, **adjust the frame pointer**
- Callee Function Executes** (during that time, the callee may invoke other functions, but this can be abstracted away)
- Callee Function Returns**
 - Callee: place the return result in a register (if applicable, usually a dedicated return register)

2. Callee: **restore saved registers and frame pointer**, restore the saved stack pointer

5. **Transfer of Control from Callee back to Caller using saved PC:**

1. Caller: uses the return result (if applicable)

2. Caller: continues execution of the program

```
void f(int a, int b)
{
    int c;

    a = 123;
    b = 456;
    c = g(a, b);
    ...
}

int g(int i, int j)
{
    int a;

    a = i + j;
    return a * 2;
}
```

local var "a"

123

456

Saved SP

Return PC

Stack Frame for f()

New SP

Old SP

Parameters

Register Spilling

- Register Spilling:** When a function has more arguments than the number of registers, the extra arguments are spilled to the stack
- Even if neither function is short of registers, the caller and callee may use the same registers, so we need to save the registers and restore them after the call.
- There are two methods to save the registers:
 - Callee-saved:** the callee saves the old values of the registers into its stack frame and restores them after the call
 - Caller-saved:** the caller saves needed registers into its stack frame and restores them after the call
- CS2106 convention: all registers are callee-saved unless otherwise specified

2.3 Heap Memory

Heap Memory

- The **heap** is a region of memory used to store dynamically allocated data.
- Dynamically allocated data in C: malloc and free
- Dynamically allocated data cannot be stored in:
 - Data region: size must be known at compile time
 - Stack: the data may outlast the function call
- Heap management is harder than stack management as allocation and deallocation can happen in arbitrary orders, leading to fragmentation.

2.4 Process States

Process State Model

New

Ready

Running

Blocked

Terminated

create

admit

switch: scheduled

switch: release CPU

event occurs

event wait

exit

- A process can be in one of the following states:
 - New:** process has been created but not fully initialised/admitted to the system
 - Ready:** process is waiting to execute
 - Running:** process is currently executing
 - Blocked:** process is waiting for an event to occur (e.g. I/O completion, signal)
 - Terminated:** process has finished execution

Multi-Process Management

- With one CPU core, at most one process can be running at a time
- With m CPU cores, at most m processes can be running at a time
- CS2106 convention: assume one CPU core unless otherwise specified
- Different processes may be in different states at the same time

Process Queues

admit

ready queue

switch by scheduler

Running Process

exit

release cpu

blocked queue

event wait

CPU

- The OS maintains a queue of processes for each state
- Ready Queue:** processes that are ready to be scheduled to run on the CPU
- Blocked Queue:** processes that are waiting for an event to occur
- More than one process can be in the ready and blocked queues at the same time
- There may be separate queues for different types of blocked processes (e.g. I/O blocked, signal blocked)

2.5 Process Control Block (PCB)

A Process Control Block (PCB) or Process Table Entry is a data structure describe the execution context for a process, maintained by the Kernel.

PCB₁

PCB₂

PCB₃

.....

Process Table

PC, FP, SP,

GPRs

Memory Region Info

PID

Process State

Process Control Block

Text

Data

Heap

Stack

Memory Space of a Process

6. System call handler ended by restoring CPU state and return to user mode.

7. Library call returns the result to the user program.

PCB₁

PCB₂

PCB₃

.....

Process Table

PC, FP, SP,

GPRs

Memory Region Info

PID

Process State

Process Control Block

Text

Data

Heap

Stack

Memory Space of a Process

Exception and Interrupt

- Exception:**
 - Occurs during the execution of a program
 - Synchronous in nature (i.e. caused by the program itself)
 - E.g. divide by zero error or a page fault
- Interrupt:**
 - Occurs during the execution of a program
 - Asynchronous in nature (i.e. caused by external events)
 - E.g. hardware interrupt or a software interrupt

```
void f()
{
    ...
    Statement S1;
    ...
}

void handler()
{
    1. Save Register/CPU state
    2. Perform the handler routine
    3. Restore Register/CPU
    4. Return from interrupt
}
```

1. Exception/Interrupt occurs:

- Control transfer to a handler routine automatically

2. Return from handler routine:

- Program execution resume
- May behave as if nothing happened

2.7 A Case Study in Unix

Process Abstraction

In Unix, an entry in PCB consists of:

1. Identification:

- PID: Process ID

2. Information:

- Process State: Running, Sleeping, Stopped, Zombie, etc.
- Parent PID: PID of the parent process
- Cumulative CPU time: Total CPU time the process has consumed
- etc.

2.7.1 Process Creation

fork()

fork() creates a new process by duplicating the current executable image.

- A forked process
 - Shares the same code, address space, open files, etc.
 - Differs only in its own process ID (PID) and parent process ID (PPID).
- Returns 0 in child process and the PID of the child process in the parent process.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
```

```
int main() {
    int result = fork();
    if (result != 0) { // 0 means child process, non-zero means parent process
        printf("P:My Id is %i\n", getpid());
        printf("P:Child Id is %i\n", result);
    } else {
        printf("C:My Id is %i\n", getpid());
        printf("C:Parent Id is %i\n", getpid());
    }
}

// STDOUT:
// P:My Id is 1234
// P:Child Id is 5678
// C:My Id is 5678
// C:Parent Id is 1234

exec()
```

init() and Process Tree

init() is the first process created by the kernel.

- It is the parent of all other processes.
- It is a system process that is responsible for starting the system and managing the system services.

2.7.2 Process Termination

exit()

exit(int status) terminates the current process with a status code.

- Status code is an integer passed to the parent process.
 - If the status code is 0, the process terminated successfully.
 - If the status code is non-zero, the process terminated abnormally.
- On process exit():
 - Process state is set to **Zombie** state.
 - While most system resources are released by the OS, some basic process resources are not releasable:
 - PID, PPID, etc.
 - Process accounting information is not releasable.

Orphan vs Zombie Processes

- Orphan Process:**
 - A process whose parent process has terminated.
- Child termination sends signal to init, which utilizes wait() to cleanup
- Zombie Process:**
 - A process whose parent process has not yet collected its exit status.
 - Can fill up process table and may need a reboot to clear the table on older Unix implementations

creation fork()

Ready

Running

Stopped

Zombie

Termination exit()

context switch

waiting for resource, e.g., I/O

Suspended

resource ready

STOP signal

CONT signal

Implementation issues with fork()

- Memory duplication is expensive.
- Copy-on-write optimization can be used to avoid unnecessary memory duplication.